

Twenty-Fourth Implementation Iteration: Request Purpose and Metadata with Descriptive Metadata in the Base Request Model

Software Design / Iteration Report

Name: Najib Attar

Date: May 2026

1. Iteration Goal

The goal of this twenty-fourth implementation iteration is to make each request more descriptive by adding general metadata to the base request model.

Before this iteration, requests already stored technical allocation fields such as requester, requested space, participant count, required feature, required building, time information, status, priority, rejection reason, and lifecycle history. These fields were sufficient for processing and rule evaluation, but they did not clearly explain why a request existed from a human perspective.

In this iteration, two descriptive metadata fields are added to the abstract `Request` base class:

- `title`
- `purpose`

Examples include a title such as `BLG101 Midterm Exam` and a purpose such as `Exam`, or a title such as `Project Meeting` and a purpose such as `Meeting`. This provides a better foundation for later exam-related work without introducing a dedicated `ExamRequest` class yet.

2. Changes from Iteration 23

Compared with the twenty-third iteration, the following changes were introduced:

- Added `title` and `purpose` fields to the `Request` base class
- Added `getTitle()` and `getPurpose()` access methods
- Updated `OneTimeRequest`, `RecurringRequest`, and `InvalidRequest` so their constructors forward metadata to the base request class

- Updated `RequestFactory` so it receives and forwards request metadata during object creation
- Updated `DataController` so it parses optional `title` and `purpose` columns from `requests.csv`
- Added safe default values for older rows that do not contain metadata:
 - `title = Untitled Request`
 - `purpose = General`
- Updated `ConsolePrinter` to display title and purpose
- Updated `RequestResultWriter` to export title and purpose
- Extended the sample `requests.csv` data with descriptive metadata

This iteration does not introduce `ExamRequest`, multi-room allocation, resource-sharing behavior, or new allocation algorithms.

3. Scenario

The prototype begins by loading users, spaces, requests, and configuration data through the existing data access flow. During request loading, `DataController` still reads CSV fields and resolves user and space references. However, it now also reads optional metadata fields for request title and purpose.

After the request row has been parsed, `DataController` passes the metadata together with the existing technical fields to `RequestFactory`. The factory creates the appropriate request object and forwards the metadata to the base `Request` constructor. If metadata is missing because an older request row uses the previous CSV format, the request still loads safely and receives default values.

Once requests are created, the rest of the system behaves exactly as before. Requests are evaluated by the rule engine, processed by the selected allocation strategy, printed to the console, and exported to `request_results.csv` together with their lifecycle history.

4. Modules and Their Tasks

A. Request Base Class Module

- Store common request data shared by all request types
- Now also store descriptive metadata through `title` and `purpose`
- Provide shared access methods for technical and descriptive request information

B. Concrete Request Module

- Continue representing one-time, recurring, and invalid request forms
- Forward title and purpose into the base request constructor
- Preserve their existing request-type-specific behavior

C. RequestFactory Module

- Continue centralizing request object creation
- Accept and forward metadata together with existing request fields
- Preserve invalid request creation for malformed or unsupported request rows

D. DataController Module

- Continue parsing request CSV rows and resolving user/space references
- Read optional metadata columns when present
- Provide safe defaults when metadata columns are missing

E. Reporting and Export Module

- Display title and purpose in console output
- Export title and purpose in `request_results.csv`
- Preserve status, rejection reason, and lifecycle history output

5. Iteration Comparison

Iter.	Focus	Added / Modified	Design bability	Responsi-	Prototype Effect
21	Request priori- tization	PriorityAllocation	Strategy	Implemented a separate strategy for deciding processing order.	Requests were no longer processed only by insertion order; priority affected execution.
22	Polymorphic users	User hierarchy and UserFactory		Moved user-specific data such as role, priority, and authorization into user classes.	System behavior became user-driven and easier to extend for different user types.
23	Request cre- ation	RequestFactory		Separated request construction from DataController.	The controller became cleaner and less responsible for object creation.
24	Request meta- data	title and purpose in Request		Extended requests with descriptive information for future use cases.	Requests became clearer, more informative, and better prepared for exam-oriented extensions.

6. Updated Class Diagram

Figure 1 shows the updated class diagram. The main change in this iteration is the addition of `title` and `purpose` to the base `Request` class together with the new accessor methods. The diagram also continues to show `RequestFactory` creating `OneTimeRequest`, `RecurringRequest`, and `InvalidRequest` objects.

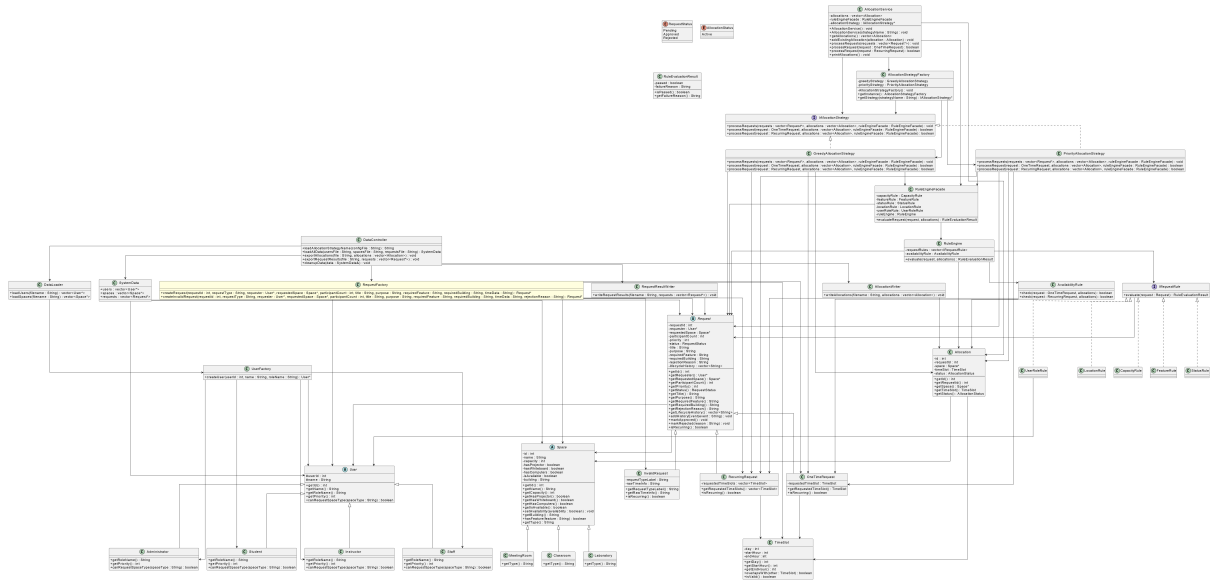


Figure 1: Updated Class Diagram with Request Metadata

7. Updated Workflow Diagram

Figure 2 shows the updated workflow. During request loading, `DataController` now parses optional title and purpose fields before delegating object creation to `RequestFactory`. The rule evaluation and allocation strategy flow remain unchanged.

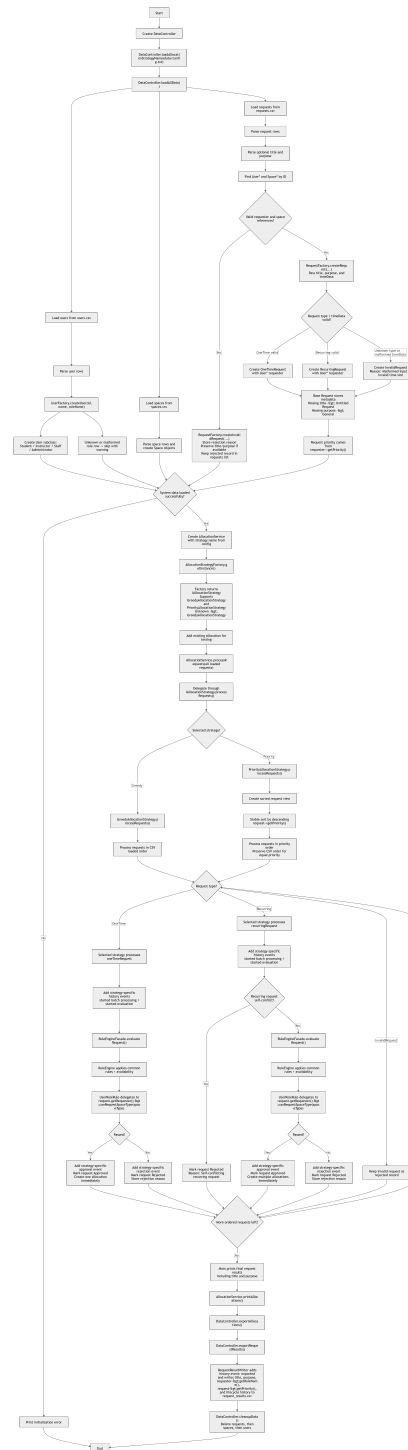


Figure 2: Workflow with Request Metadata Parsing and Creation

8. Prototype Implementation

The C++ prototype was extended by adding descriptive metadata directly to the abstract `Request` base class. This means that all request types can now carry a readable title and a general purpose without duplicating those fields in each concrete request subclass.

The constructor chain was updated so that `OneTimeRequest`, `RecurringRequest`, and `InvalidRequest` pass metadata into the base request constructor. `RequestFactory` was also updated so that it accepts title and purpose and forwards them during request creation. This keeps the metadata change aligned with the existing factory-based request construction design introduced in the previous iteration.

On the loading side, `DataController` now reads optional metadata columns from `requests.csv`. If the current row contains title and purpose, those values are used directly. If the row uses the older CSV structure, the request still loads correctly and receives the safe defaults `Untitled Request` and `General`. This preserves backward compatibility with earlier request data.

Console and CSV reporting were also updated. Request outputs now show not only technical request details but also the human-readable title and purpose. This makes the request results easier to understand and prepares the prototype for future work in which exam-related requests may need clearer semantic identification.

9. Summary

This twenty-fourth implementation iteration adds general request metadata through `title` and `purpose`. The change is implemented at the `Request` base level so that it applies uniformly to one-time, recurring, and invalid requests.

The iteration improves the descriptive quality of request data while preserving the current architecture, request creation flow, rule engine behavior, allocation strategies, lifecycle history, and export behavior. Most importantly, it establishes a clean foundation for future exam-related work without prematurely introducing an exam-specific request type.